

Code Analysis Report

1. Executive Summary

The project is an AI Resume Checker application. It supports account creation, resume upload, Gemini-based single-resume ATS analysis, saved scan history, dashboard analytics, profile management, settings, and multi-resume candidate ranking.

The active runtime path is:

React/Vite frontend -> FastAPI backend -> MongoDB

There is also an Express backend in `backend/server.js`, but the current root scripts run the Python FastAPI backend in `backend/main.py`.

2. Technology Stack

Layer	Technology
Frontend	React 19, Vite, React Router, Framer Motion
UI and charts	Tailwind-style CSS classes, React Icons, Recharts
Backend	Python, FastAPI, Uvicorn
Database	MongoDB with Motor async driver
Authentication	Email/password with bcrypt password hashing
AI analysis	Gemini API called from frontend
NLP ranking	NLTK, scikit-learn TF-IDF, cosine similarity, pypdf
Alternate backend	Express, Mongoose, Multer, bcryptjs

3. Frontend Code Analysis

frontend/src/App.jsx

This file defines the main frontend routes:

/	Landing page
/login	Login
/signup	Signup
/forgot-password	Forgot password page
/dashboard	User dashboard
/upload	Single resume upload
/loading	AI analysis loading flow
/result	Single resume analysis report
/analytics	Charts and keyword analytics
/suggestions	Resume bullet suggestions
/profile	User profile and saved resumes
/settings	App settings and account deletion
/ranking	Multi-candidate ranking

`App.jsx` wraps everything in `ResumeProvider`, so user and resume analysis data are available across pages.

frontend/src/context/ResumeContext.jsx

This context stores:

- `resumeData`: the latest Gemini analysis result or loaded saved report.
- `user`: the logged-in user object.
- `setResumeData`: updates the active report.
- `setUser`: updates logged-in state.

The `user` value is persisted in `localStorage`, so refreshes keep the login session on the client side.

Authentication Pages

`Login.jsx` sends credentials to:

```
POST http://localhost:5000/api/login
```

If successful, it stores the returned user in `ResumeContext` and navigates to `/dashboard`.

`Signup.jsx` sends:

```
POST http://localhost:5000/api/signup
```

After signup, it redirects to login with a success message.

Single Resume Pages

`UploadResume.jsx` collects:

- resume file
- job description
- email/password if the user is not logged in
- existing user id if logged in

It passes those values to `LoadingScreen.jsx` through React Router state.

`LoadingScreen.jsx` does the most important single-resume work:

1. Reads the uploaded file using `FileReader`.
2. Converts it to base64.
3. Builds a strict JSON prompt for Gemini.
4. Calls the Gemini API directly from browser JavaScript.
5. Parses the JSON result.
6. Stores the result in `ResumeContext`.

7. Uploads the file plus parsed AI result to FastAPI `/api/upload`.

8. Navigates to `/result`.

`ResultPage.jsx` renders the analysis with:

- match score
- ATS score
- recruiter score
- skill radar chart
- missing keywords
- AI suggestions
- resume preview when the file is available

Dashboard And History Pages

`Dashboard.jsx` loads saved scans with:

```
GET http://localhost:5000/api/resumes?userId=<userId>
```

It computes summary metrics on the frontend and renders a performance chart.

`ProfilePage.jsx` shows account information, uploaded resumes, and scan reports. It can fetch full saved reports, download saved resume files, update profile fields, and delete resumes.

`SettingsPage.jsx` stores UI settings in `localStorage`, toggles dark mode, lists database resumes, and can delete the logged-in account.

Candidate Ranking Page

`RankingPage.jsx` supports multi-resume screening:

1. User uploads one or more files.
2. User pastes a job description.
3. It sends multipart form data to:

```
POST http://localhost:5000/api/rank
```

1. It renders ranked candidates with scores, skills, missing skills, and explainability details.

4. Backend Code Analysis

`backend/main.py`

This is the active API server. It creates a FastAPI app, enables permissive CORS, and defines all backend routes.

Main responsibilities:

- account signup/login

- password hashing and verification
- resume upload and storage
- saved report retrieval
- resume deletion
- account deletion
- profile update
- NLP candidate ranking

Important functions:

```
hash_password(password)
verify_password(password, hashed_password)
signup(user_data)
login(user_data)
upload_resume(...)
get_resumes(userId)
get_resume_by_id(id)
delete_resume(id)
delete_account(id)
update_profile(id, profile_data)
rank_resumes(jobDescription, resumes)
rank_health()
```

backend/database.py

This file creates the MongoDB client:

```
MONGO_URI = env MONGO_URI or mongodb://localhost:27017/
DB_NAME = ai-resume-checker
```

It exposes:

```
get_users_collection()
get_resumes_collection()
```

backend/models.py

This file contains:

- `serialize_doc()`: converts MongoDB values into JSON-safe values.
- `UserSignup`: signup request schema.
- `UserLogin`: login request schema.
- `UserUpdate`: editable profile schema.

`serialize_doc()` is important because MongoDB `ObjectId`, `datetime`, and raw file bytes cannot be returned directly as JSON.

backend/nlp_engine.py

This is the ranking engine.

Main functions:

```
extract_text_from_pdf(pdf_bytes)
preprocess_text(text)
extract_features(text)
compute_tfidf_similarity(job_description, resumes)
rank_candidates(job_description, candidates)
```

Ranking pipeline:

1. Extract PDF text.
2. Lowercase text.
3. Remove URLs, emails, numbers, punctuation, and extra spaces.
4. Tokenize with NLTK.
5. Remove English and resume-specific stopwords.
6. Stem words with PorterStemmer.
7. Create TF-IDF vectors with unigrams and bigrams.
8. Compute cosine similarity between job description and each resume.
9. Extract skill overlap.
10. Calculate final composite score.
11. Sort candidates by score.

The final score is based on:

```
80% TF-IDF similarity + 20% skill coverage bonus
```

backend/server.js

This is an Express/Mongoose backend that duplicates many routes from the FastAPI backend. It uses:

- Express
- Mongoose
- Multer
- bcryptjs

It is useful as a reference or older implementation, but it is not the backend started by the root `npm run dev` command.

5. Database Analysis

The application uses MongoDB database:

```
ai-resume-checker
```

Collections:

```
users
resumes
```

User documents contain:

```
email
password
name
title
createdAt
updatedAt
```

Resume documents contain:

```
userId
filename
contentType
data
parsedData
atsScore
matchScore
createdAt
updatedAt
```

The resume file itself is stored as bytes in MongoDB. The report data from Gemini is stored as `parsedData`.

6. Key Code Strengths

- The app has a clear user-facing workflow: upload, analyze, view report, save history, compare candidates.
- FastAPI routes are straightforward and easy to test.
- Passwords are hashed before storage.
- MongoDB serialization handles common JSON issues such as `ObjectId`, `datetime`, and binary data.
- Candidate ranking is explainable because it returns matched skills, missing skills, TF-IDF score, and skill bonus.
- React pages are separated by feature, which makes the UI flow easy to follow.
- The ranking engine is independent from the UI, so it can be tested separately.

7. Risks And Issues

High Priority

- The Gemini API key is hard-coded in `frontend/src/pages/LoadingScreen.jsx`. Anyone with the built frontend can see and use it. Move Gemini calls to the backend and keep the key in an environment variable.
- Frontend calls Gemini directly. This makes error handling, key protection, rate limiting, and request auditing difficult.

- CORS is set to allow all origins in `backend/main.py`. This is acceptable for local development, but should be restricted in production.

Medium Priority

- The UI accepts DOCX files in multiple places, but `nlp_engine.extract_text_from_pdf()` only extracts PDF text. Candidate ranking for DOCX files may return empty text and weak scores.
- Authentication is client-state based. There is no token, session, or server-side authorization check protecting resume access.
- `GET /api/resumes` can return all resumes if `userId` is omitted. This should be restricted for a real multi-user app.
- `backend/server.js` and `backend/main.py` duplicate backend concepts. Keeping both can confuse future development.
- Some displayed text contains encoding artifacts such as `â€¢`, `â€”`, and similar symbols. These should be cleaned.

Lower Priority

- `models.json` is empty.
- There are generated files and dependency folders inside the repo workspace, including `node_modules`, `frontend/node_modules`, and `backend/node_modules`.
- `nlp_engine.py` attempts NLTK downloads at import time. This can slow startup or fail in offline environments.
- The project has no visible automated tests.

8. Recommended Improvements

1. Move Gemini API usage into FastAPI.
2. Store the Gemini API key in an environment variable.
3. Add JWT or session-based authentication.
4. Require authenticated access for resume list, resume detail, delete, profile update, and account delete routes.
5. Add DOCX extraction with a library such as `python-docx`.
6. Remove or clearly archive `backend/server.js` if FastAPI is the official backend.
7. Add backend tests for auth, upload, resume retrieval, deletion, and ranking.
8. Add frontend API configuration so URLs are not hard-coded.
9. Add file size/type validation on both frontend and backend.
10. Clean encoding artifacts in UI strings and existing reports.

9. Development Entry Points

Use these files first when modifying the project:

Task	Start Here
Add or change routes	frontend/src/App.jsx
Change shared user/report state	frontend/src/context/ ResumeContext.jsx
Change single resume analysis flow	frontend/src/pages/ LoadingScreen.jsx
Change report display	frontend/src/pages/ ResultPage.jsx
Change candidate ranking UI	frontend/src/pages/ RankingPage.jsx
Change backend APIs	backend/main.py
Change database connection	backend/database.py
Change JSON serialization/schema	backend/models.py
Change ranking algorithm	backend/nlp_engine.py

10. Overall Conclusion

The project is functional and has a sensible feature split between frontend pages, backend APIs, MongoDB persistence, and NLP ranking. The biggest architectural change needed is moving Gemini analysis out of the browser and into the backend. After that, authentication and authorization should be strengthened so saved resumes and reports are protected per user.